

Доказательства корректности, как мощный механизм гарантий качества

или всё ли можно проверить тестами?

Арсений Жижелев, Scala-архитектор, Праймтолк

22.11.2025

Тезисы

1. Качество программы - наша цель.
2. Тесты несовершенны. Иногда недостаточны, а иногда избыточны.
3. Типы в некоторых случаях позволяют обойтись без тестов.
4. Формулировки требований сильно влияют на выбор тесты/типы (сейчас bias в сторону тестов).

Иначе говоря,

```
Как писать программы,
чтобы гарантировать, что они будут работать правильно,
но при этом писать поменьше тестов?
```



Качество программ (1)

Качественная программа — (1) достигает (2) требуемого результата (3) с соблюдением заданных ограничений и (4) минимизирует целевую функцию.

Где здесь тесты? Где здесь типы?

Программа + тесты (2)

Типичная backend Scala-программа

```
def handlerCe(deps: Deps): Request ⇒ F[Either[Error, Response]]
либо
val handlerZio: Request ⇒ ZIO[Deps, Error, Response]
```

Тесты

```
val testCases = List(
  Request( ... ) -> Right(Response( ... )),
  Request( ... ) -> Left(Error( ... )),
  ...
)

testCases.foreach { case (request, expected) ⇒
  val actual = handler(request) // с точностью до F[_]
  assert(actual == expected)
}
```

Проблемы тестов

1. Мощность тестов << мощности программы.
2. Моки.
3. Карго-культ (что именно тестируем?).

Типы (3)

0. Компилятор, статическая система типов.
1. Дженерики (параметрический полиморфизм).
2. Тайп-классы.
3. ADT - Алгебраические типы данных.
4. newtype'ы.
5. Refined-типы.
6. Литералы (синглетонные типы).
7. Квитанции эффектов (`newtype Unit`).

Далее - несколько слайдов по известным применениям типов.



Типы (3.1): компилятор (vs интерпретаторы с динамической типизацией)

Типы:

- каждая строка кода проходит компиляцию
- опечатки
- неправильные аргументы функций
- несовместимые типы в одном выражении

Тесты, которые не нужны:

- 100% покрытие не требуется
- проверка входных данных на соответствие Int/Double
- проверка списка аргументов функции

Типы (3.2): компилятор (vs интерпретаторы с динамической типизацией)

Типы:

- каждая строка кода проходит компиляцию
- опечатки
- неправильные аргументы функций
- несовместимые типы в одном выражении

Тесты, которые не нужны:

- 100% покрытие не требуется
- проверка входных данных на соответствие Int/Double
- проверка списка аргументов функции

Типы (3.3): Дженерики

Типы:

- некоторые функции определены типами:

```
def f[A](a: A): A = ???
val f: [A] ⇒ A ⇒ A = ???

val identity: [A] ⇒ A ⇒ A = [A] ⇒ (a: A) ⇒ a
```

- универсальные функции `map`, `flatMap`, `foldLeft`, `traverse`, ... - написаны один раз, используются многократно

Тесты, которые не нужны:

- для каждого типа данных
- меньше кода — меньше тестов



Типы (3.4): АДТ (алгебраические типы данных)

Типы:

- точная передача семантики (bool -> enum)
- закрытый список вариантов (enum'ы, sum-типы)
- Either[Error, A] вместо исключений

Тесты?

- тесты исключений (тесты обработки ошибок)
- тесты ветвлений

Не тестируемые проблемы

- расширение enum при рефакторинге
- непредусмотренная обработка каких-либо случаев

Типы (3.5): Явный Null

Типы:

```
val x: String | Null = null // ok
-Yexplicit-nulls
```

Тесты?

- проверка на null всех N функций и всех комбинаций M аргументов O(N*2^M)

```
def foo(a: A|Null, b: B|Null): Int = (a, b) match
  case (null, null) ⇒ 0
  case (null,    _) ⇒ 1
  case (   _, null) ⇒ 2
  case _            ⇒ 3
def bar(a: A, b: B): Int = 3
```

Типы (3.6): * Квитанции эффектов (доказательства)

(Малоизвестный приём доказательства выполнения эффекта.)
Доказательство сохранения в БД:

```
object Db:
  opaque type Identifier[A] = DbGeneratedId

  def create[F[_]: Storage, A](a: A): F[Identifier[A]] =
    Storage[F].insertReturningId(a)
      .map(id ⇒ id: Identifier[A])

object TaskService:
  opaque type TaskId = Identifier[Task]

  def createTask[ ... ](taskDefinition: Task): F[TaskId] =
    Db.create(taskDefinition)
```

Доказательство отправки уведомления:

```
object RemoteSystem:
  sealed trait Notified
  def notify(notification: Notification): F[Notified] =
    ...
    .as(new Notified {})
```



Квитанции эффектов (habr)

Типы (3.6.2): * Квитанции эффектов

Применения квитанций эффектов:

- доказательство отправки уведомлений;
- доказательство аудита безопасности;
- доказательство выполнения проверки доступа;

Исключаемые тесты:

- проверка отправки уведомлений;
- проверка с помощью моков вызова эффектов;

Типы (3.6.3): * Агрегирование квитанций

- Scala 3 Tuple
- HList
- Scala 3 TypeSet (https://github.com/Primetalk/type-sets)

```
type Notified1 = RemoteSystem.Notified
type Notified2 = RemoteSystem.Notified

trait Proof[A]

given [H, T <: Tuple](using Proof[H], Proof[T]): Proof[H *: T] = ???

case class Proved[A, P](value: A, proof: P)
```

Типы как теоремы (3) Изоморфизм Карри-Говарда

Основой доказательств является связь между типизированной программой и обычной логикой.

```
type Id = [A] ⇒ A ⇒ A
val identity: Id = a ⇒ a

// Modus Ponens
// ∀A, B (A -> B) & A -> B
def modusPonens[A, B](f: A ⇒ B, a: A): B = f(a)
val modusPonens: [A] ⇒ [B] ⇒ ((A ⇒ B), A) ⇒ B = {
  case (f, a) ⇒ f(a)
}
```

Какие теоремы интересны в прикладных приложениях? (4)

(Суха, мой друг, теория...)

1. Работоспособность (компилируется == работает).
2. Универсальность (работает для всех входных данных, тотальность).
3. Семантическая корректность (выдает правильный результат).
4. Безопасность (роли, права доступа, аутентификация, авторизация).
5. Побочные эффекты (во внешние системы отправляются уведомления, в БД сохраняются данные, логируется то, что нужно).
6. Бизнес-логика как параметр (основная структура программы не зависит от бизнес-логики).

Бизнесовые теоремы отличаются от алгоритмических/математических теорем!!!



Формулировка требований (4)

Сравним два требования:

Требование 1:

Я, как пользователь, при нажатии на кнопку “Сохранить”, должен получить сообщение “Сохранено успешно”.

Требование 2:

При всех возможных входных данных, если программа завершилась успехом, то гарантируется, что (1) новые данные сохранены в БД; (2) отправлены обязательные уведомления во внешние системы. Если программа завершалась неудачей, то гарантируется, что (1) состояние БД не изменилось; (2) во внешние системы отправлены компенсирующие уведомления.

Заключение (4)

1. Явно описанные универсальные требования можно гарантировать только с помощью типов.
2. Выход за пределы языка (интеграция с внешними системами,) Некоторые частные ad-hoc требования остаётся проверять только тестами.
3.

Пример

Платформа доставки Клиентский процесс вызывает delivery-platform.

[] хорошо бы убрать отсылку на Т-Банк

[] отсутствие идемпотентности! [] несовместимость контракта! (Отслеживается на уровне linter'а OpenAPI)

[] интеграционный тест - не решение проблемы, нужно компилировать систему из нескольких систем

[] DDD - ? модель - что с обратной совместимости?

[] тесты надёжности [] тесты нагрузочные